

Advanced approximate and symbolic computing

Tamás Vinkó

University of Szeged



Orthogonal transformations IV.

QR-algorithm

QR-algorithm

- Starting from matrix $\mathbf{A}_1 \in \mathcal{R}^{n \times n}$ we introduce the recurrent definitions

$$\mathbf{A}_k = \mathbf{Q}_k \mathbf{R}_k \quad (1)$$

$$\mathbf{A}_{k+1} = \mathbf{R}_k \mathbf{Q}_k \quad (2)$$

- This is called **QR**-algorithm.
- Since the QR decomposition exists for *any* matrix $\mathbf{A}_k$, the elements of the above defined series can be calculated the previously studied methods for **QR** factorization.

One of the most important properties of the matrix series is stated in the following lemma.

Lemma 7.1 *Matrix $\mathbf{A}_k$ is similar to matrix $\mathbf{A}_1$.*

Proof. We show that $\mathbf{A}_k$ is similar to matrix $\mathbf{A}_{k+1}$.

$$\begin{aligned}\mathbf{A}_k &= \mathbf{Q}_k \mathbf{R}_k \\ \mathbf{A}_{k+1} &= \mathbf{R}_k \mathbf{Q}_k\end{aligned}$$

Reordering the first equation we obtain: $\mathbf{Q}_k^T \mathbf{A}_k = \mathbf{R}_k$, which can be substituted into the second equation:

$$\mathbf{A}_{k+1} = \mathbf{Q}_k^T \mathbf{A}_k \mathbf{Q}_k,$$

which shows that indeed $\mathbf{A}_{k+1} \sim \mathbf{A}_k$. ■

we give a more detailed lemma and proofs in the lecture notes

- [illegible]

Proposition 7.2 (*Convergence theorem of the QR-algorithm*) If matrix $\mathbf{A}_1 \in \mathcal{R}^{n \times n}$ fulfills the following conditions:

1. indexing the eigenvalues of matrix $\mathbf{A}_1$ in this way

$$|\lambda_1| > |\lambda_2| > \cdots > |\lambda_n| > 0$$

2. we can define such matrix $\mathbf{X}$ which diagonalizes matrix $\mathbf{A}_1$:

$$\mathbf{X}^{-1}\mathbf{A}_1\mathbf{X} = \mathbf{D} = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n),$$

moreover, there exists the triangular decomposition $\mathbf{X}^{-1} = \bar{\mathbf{L}}\bar{\mathbf{R}}\dots$

... then the matrix series $\mathbf{A}_k$ which is produced by the **QR**-algorithm is convergent:

$$\lim_{k \rightarrow \infty} \mathbf{A}_k = \begin{pmatrix} \lambda_1 & * & * & \cdots & * \\ 0 & \lambda_2 & * & \cdots & * \\ 0 & 0 & \lambda_3 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & * \\ 0 & 0 & \cdots & 0 & \lambda_n \end{pmatrix}.$$

Proof. – ■

How can we give the conditions of this theorem?

Interesting to note: the QR-decomposition works for **any** matrix

Properties, complexity

What are the advantages of **QR**-algorithm?

- (1) It is stable (with respect to rounding errors).
- (2) In general, the cost of one step is $O(n^3)$.
- (3) It keeps the symmetry: if we assume that $\mathbf{A}_k$ is symmetric, then

$$\mathbf{A}_{k+1} = \mathbf{R}_k \mathbf{Q}_k = \mathbf{Q}_k^T \mathbf{A}_k \mathbf{Q}_k, \text{ hence } (\mathbf{A}_{k+1})^T = \left(\mathbf{Q}_k^T \mathbf{A}_k \mathbf{Q}_k \right)^T = \mathbf{A}_{k+1}.$$

- (4) It keeps the Hessenberg form.

What is a Hessenberg form?

Transforming into upper Hessenberg form by Householder method

Theorem 7.3 Any matrix $\mathbf{A} \in \mathcal{R}^{n \times n}$ can be transformed into upper Hessenberg form by using finite many orthogonal similarity transformations. More precisely, we can define such series of reflection matrices $\mathbf{H}_1, \mathbf{H}_2, \dots, \mathbf{H}_{n-2}$ such that the recurrence

$$\mathbf{A}_0 \stackrel{\text{def}}{=} \mathbf{A}, \quad \mathbf{A}_j \stackrel{\text{def}}{=} \mathbf{H}_j \mathbf{A}_{j-1} \mathbf{H}_j \quad j = 1, 2, \dots, n-2$$

gives such matrix $\mathbf{A}_j$ which $1, 2, \dots, j$ -th column vectors are upper Hessenberg, i.e. $a_{ik}^{(j)} = 0$, if $1 \leq k \leq j$ and $i > k + 1$.

Specifically, matrix $\mathbf{A}_{n-2}$ is upper Hessenberg.

Transforming into upper Hessenberg form by Householder method

- Since, by the previous theorem, $n - 2$ similarity transformation makes matrix $\mathbf{A}_{n-2}$ into upper Hessenberg form,
- and the cost of one similarity transformation is $O(n^2)$,
- we can state the following.

Corollary 7.4 *Any matrix $\mathbf{A} \in \mathcal{R}^{n \times n}$ can be transformed into upper Hessenberg form by $n - 2$ reflection matrices using similarity transformations with $O(n^3)$ operations.*

Transforming into upper Hessenberg form by Householder method

- We know that orthogonal similarity transformations keep symmetry,
- hence if the starting matrix $\mathbf{A}$ is symmetric, then the resulting upper Hessenberg matrix will also be symmetric,
- more precisely: tridiagonal.

Corollary 7.5 *Any symmetric matrix $\mathbf{A} \in \mathcal{R}^{n \times n}$ can be transformed into tridiagonal form by $n - 2$ reflection similarity transformations. The cost is $O(n^3)$, but exploiting symmetry we can roughly halve the total computational cost (compared to the general case).*

Transforming into upper Hessenberg form by Givens method

Similarly, we can also use rotation matrices, as it follows.

Theorem 7.6 *Any matrix $\mathbf{A} \in \mathcal{R}^{n \times n}$ can be transformed into upper Hessenberg form using finite many similarity rotation transformations. More precisely one can give series of rotation matrices*

$\mathbf{G}_{2,3,\theta_{2,3}}, \mathbf{G}_{2,4,\theta_{2,4}}, \dots, \mathbf{G}_{2,n,\theta_{2,n}}, \mathbf{G}_{3,4,\theta_{3,4}}, \mathbf{G}_{3,5,\theta_{3,5}}, \dots, \mathbf{G}_{3,n,\theta_{3,n}}, \dots, \mathbf{G}_{n-1,n,\theta_{n-1,n}}$ such that

$$\mathbf{B} \stackrel{\text{def}}{=} \mathbf{G}_{n-1,n,\theta_{n-1,n}} \cdots \mathbf{G}_{3,n,\theta_{3,n}} \cdots \mathbf{G}_{3,5,\theta_{3,5}} \mathbf{G}_{3,4,\theta_{3,4}} \mathbf{G}_{2,n,\theta_{2,n}} \cdots \mathbf{G}_{2,4,\theta_{2,4}} \mathbf{G}_{2,3,\theta_{2,3}} \mathbf{A} \mathbf{G}_{2,3,\theta_{2,3}}^T \mathbf{G}_{2,4,\theta_{2,4}}^T \cdots \mathbf{G}_{2,n,\theta_{2,n}}^T$$

is upper Hessenberg form.

I know, some part is missing here, but it is not that important. It is only good to know that we have matrix $\mathbf{A}$ in the series.

Transforming into upper Hessenberg form by Givens method

By the previous theorem we need $O(n^2)$ similarity transformations to get the upper Hessenberg form, and one similarity transformation can be done with $O(n)$ operations, we can state the following:

Corollary 7.7 *Any matrix $\mathbf{A} \in \mathcal{R}^{n \times n}$ can be transformed into upper Hessenberg form using $O(n^2)$ rotation matrices, so this similarity transformation needs $O(n^3)$ operations.*

Since the orthogonal similarity transformations keep symmetry it holds that if the starting matrix $\mathbf{A}$ is symmetric then the resulting upper Hessenberg matrix is also symmetric, which means that it is tridiagonal.

Corollary 7.8 *Any symmetric matrix $\mathbf{A} \in \mathcal{R}^{n \times n}$ can be transformed into tridiagonal form by $O(n^2)$ rotation matrices. We need $O(n^3)$ operations, however, by exploiting symmetry, the number of operations can be halved (compared to the general case).*

QR algorithm on special matrices

The above discussed

- upper Hessenberg
- and tridiagonal

matrices we need less computational complexity.

Proposition 7.9 *The QR-algorithm keeps the upper Hessenberg form of a matrix, so if the starting matrix $\mathbf{A}_1$ is upper Hessenberg, then all matrices $\mathbf{A}_k$ are also upper Hessenberg. Thus, the complexity of one step is $O(n^2)$.*

For triangular matrices we have even better case:

Proposition 7.10 *The QR-algorithm keeps the tridiagonal form of a matrix, so if the starting matrix $\mathbf{A}_1$ is tridiagonal then matrix $\mathbf{A}_k$ is also tridiagonal. Hence the complexity of one step is $O(n)$.*

Note that these special matrices play roles in lots of applications

QR-algorithm

1. Input: an $n \times n$ -dimensional matrix $\mathbf{A}$, which satisfies the conditions of Theorem 7.2 (this is *not* known in advance, though)
2. Using Householder or Givens transformations make matrix $\mathbf{A}$ to upper Hessenberg form. Do not forget to keep the similarity here! Let this matrix be $\mathbf{A}_1$.
3. Let $i = 0$.
4. While the stopping condition does not hold do
 - a) $i = i + 1$
 - b) $\mathbf{A}_i = \mathbf{Q}_i \mathbf{R}_i$
 - c) $\mathbf{A}_{i+1} = \mathbf{R}_i \mathbf{Q}_i$
5. Output: the last matrix $\mathbf{A}_{i+1}$, which is (almost) upper triangular.

QR-algorithm - stopping condition?

- Stop after a certain number of steps.
- Bit more sophisticated method: for the elements under the main diagonal of matrix $\mathbf{A}_{i+1}$, check if their absolute sum is smaller than a prescribed tolerance.
- Note that our investigations hold for matrices which have *all real eigenvalues* (moreover, these are supposed to be different).
- As we explained, real matrix can have complex eigenvalues.

QR-algorithm - interpretation

- In case of convergence: the elements in the main diagonal of matrix $\mathbf{R}$ are the eigenvalues of matrix $\mathbf{A}$.
- Where are the eigenvectors?
 - In case of symmetric matrix $\mathbf{A}$, in case of convergence:

$$\mathbf{A}\mathbf{Q} = \mathbf{Q}\mathbf{\Lambda},$$

where $\mathbf{\Lambda}$ is a diagonal matrix and the column vectors of $\mathbf{Q}$ are the corresponding eigenvectors.

Eigenvalue/eigenvector problem

- The **QR**-algorithm finds *all* eigenvalues, usually slow
- In applications usually the absolute largest eigenvalue is needed.
 - For that it might be better to use the *power iteration*:

$$\mathbf{x}^{(k+1)} = \frac{\mathbf{A}\mathbf{x}^{(k)}}{\|\mathbf{A}\mathbf{x}^{(k)}\|} \quad (k = 0, 1, 2, \dots)$$

- here $\mathbf{x}^{(0)}$ is usually a random starting vector.
- Let $\mathbf{x} = \mathbf{x}^{(\infty)}$, then the dominant eigenvalue is:

$$\lambda_1 = \frac{\mathbf{x}^T \mathbf{A} \mathbf{x}}{\mathbf{x}^T \mathbf{x}}.$$